

**1** (5 puntos) Considere el siguiente fragmento de código escrito para un procesador con un pipeline de 5 etapas en las que se realizan las siguientes tareas:

- Etapa 1: Fetch (acceso a McaI).
- Etapa 2: Decodificación, lectura de registros, evaluación de la condición y cálculo de la dirección en las instrucciones de salto.
- Etapa 3: Ejecución y cálculo de la dirección en las instrucciones ld y st.
- Etapa 4: Acceso a datos (McaD).
- Etapa 5: Escritura en registros.

Este procesador **no** dispone de mecanismos de adelantamiento, por lo que introduce ciclos de espera para resolver las dependencias de datos.

```

(1)      add r1, r0, r0      ; cont = 0
(2)      ld r2, 0(r3)        ; r2 = DIM
(3)      add r20, r0, r0     ; i=0
(4)while: cmp r7, r20, r2    ; while (i<DIM) {
(5)      bge r7, fin         ; if i>=DIM ir a fin
(6)      nop
(7)      ld r10, 0(r5)
(8)      ld r12, 0(r6)
(9)      sub r10, r10, r12
(10)     bz r10, igual        ; if a(i) == b(i) ir a igual
(11)     nop
(12)     st r10, 0(r5)        ; else a(i) = a(i)-b(i)
(13)     add r1, r1, 1        ; cont++
(14)igual: add r20, r20, 1    ; i++
(15)     add r5, r5, 4
(16)     add r6, r6, 4
(17)     br while            ; } fin del while
(18)     nop
(19)fin:  add r20, r0, r0

```

a) A la vista del código anterior, indique **razonadamente** si es posible deducir qué mecanismo utiliza este procesador para tratar las instrucciones de salto.

b) Indique **clara y razonadamente** cuántos ciclos de espera introducirá el procesador, y entre qué instrucciones, para resolver las dependencias de datos.

c) Calcule el CPI que se obtendría en su ejecución si en el 40 % de las iteraciones se cumple la condición en la instrucción (10). Suponga para ello  $DIM=1.000$ .

d) Calcule cuántos ciclos de parada debidos a dependencias de datos debería introducir el procesador si utilizase mecanismos de adelantamiento (forwarding). Indique **claramente** entre qué etapas se realizarían los adelantamientos necesarios en este código.

e) A partir de los resultados obtenidos en el apartado anterior, reordene el código intentando minimizar los ciclos de espera y calcule el número total de ciclos debidos a dependencias de datos.

f) Indique cómo debería modificarse este código si el procesador tuviese predicción dinámica de 2 bits. Calcule el CPI para este caso suponiendo que las tablas de predicción están inicialmente invalidadas. Suponga para ello que la tasa de aciertos del predictor en la ejecución de la instrucción (10) es del 50 % y que la instrucción de salto incondicional (17) se trata, desde el punto de vista del predictor, como una de salto condicional en la que siempre se cumple la condición. Para el cálculo del CPI suponga que el procesador dispone de adelantamientos y que el código está reordenado según su solución al apartado anterior.

## SOLUCIÓN

a) El procesador utiliza salto retardado, lo que se puede deducir por el uso de una instrucción **nop** detrás de cada salto. Como los saltos se resuelven en la etapa E2 (en ella se comprueba la condición y se calcula la dirección a la que saltar), entraría en el pipeline la siguiente instrucción, que siempre se ejecutaría, por lo que la solución más sencilla para evitar que se ejecute una instrucción que no debería, es introducir una instrucción **nop** detrás de cada salto, que equivale a un ciclo de parada *software*.

b) Las dependencias que generan ciclos de espera son las que se producen entre las siguientes instrucciones:

3 - 4, 4 - 5, 8 - 9 y 9 - 10

En los cuatro casos las dependencias se dan entre dos instrucciones consecutivas, por lo que el procesador debe introducir 3 ciclos para resolverlas, ya que la escritura en registros se produce en la etapa E5 y la lectura en la etapa E2.

Además de estas dependencias, existen tres más entre las instrucciones 2-4, 7-9 y 9-12, que se resuelven sin ciclos de espera debido a los ciclos introducidos para resolver las anteriores.

c) Separando los ciclos de parada por dependencias de datos (CP\_DD), o de control (CP\_DC) y, por otro lado, las instrucciones ejecutadas (IE), se tiene:

$$IE = 3 + 1.000 \times (10 + 0,6 \times 2) + 3 = 11.206 \text{ I}$$

$$CP\_DD = 3 + 1.000 \times 9 + 3 = 9.006 \text{ c.p.}$$

$$CP\_DC = 1.000 \times 3 + 1 = 3.001 \text{ c.p.}$$

$$\text{Número total de ciclos} = 11.206 + 9.006 + 3.001 = 23.213 \rightarrow \text{CPI} = 23.213/11.206 = \mathbf{2,07}$$

d) Utilizando adelantamientos, los ciclos debidos a las dependencias de datos vistas en el apartado b) quedarían como sigue:

3 - 4 Se resolvería sin ciclos de espera con adelantamiento E3 a E3

4 - 5 y 9 - 10 Se reducirían en ambos casos a 1 ciclo de espera con adelantamiento E3 a E2

8 - 9 Se reduciría a 1 ciclo de espera con adelantamiento E4 a E3

Se mantendrían sin ciclos de espera las dependencias 2 - 4 y 7 - 9, en las que se utilizaría adelantamiento E4 a E3

e) Las únicas dependencias que se pueden resolver reordenando el código serían:

8-9 Poniendo entre ambas, por ejemplo, la instrucción 14

9-10 Poniendo entre ambas la instrucción 16

Solo quedaría el ciclo debido a la dependencia entre 4-5, por lo que:  $CP\_DD = 1.000 \times 1 + 1 = \mathbf{1.001 \text{ c.p.}}$

El código resultante, del que únicamente se muestra la parte reordenada, sería el siguiente:

```
(8)      ld r12, 0(r6)
(14)     add r20, r20, 1
(9)      sub r10, r10, r12
(16)     add r6, r6, 4
(10)     bz r10, igual
(11)     nop
(12)     st r10, 0(r5)
(13)     add r1, r1, 1
(15)igual: add r5, r5, 4
(17)     br otro
```

f) Se deberían eliminar las instrucciones **nop**, que son innecesarias con predicción, ya que el procesador introduciría automáticamente los ciclos de espera necesarios, únicamente en los fallos de predicción. En este caso, como la predicción se realiza en E1 y la comprobación de la condición se hace en E2, sería en esta etapa donde se comprobaría si se acertó o no en la predicción, por lo que se introduciría 1 ciclo de parada en caso de fallo y cero en caso de acierto.

A continuación se muestra el comportamiento de las tres instrucciones de salto en cuanto al número de fallos de predicción:

(10) Como falla el 50 % de las veces, se producirán **500 fallos**

(17) La primera vez que se ejecuta falla en la predicción, ya que como las tablas del predictor están inicialmente invalidadas, se predice No Saltar, cuando hay que hacerlo. Si suponemos que una vez detectado el fallo se introduce su información en las tablas poniendo los bits a 10, no se producirán más fallos → **1 fallo**

(5) Solo falla al salir del while, cuando la predicción es No Saltar → **1 fallo**.

En total se producen 502 fallos →  $CP\_DC = 502 \text{ c.p.}$

$$\text{CPI} = 11.206 + 1.001 + 502/11.206 = 12.709/11.206 = 1,13$$

**2** (3 puntos) Sea un multiprocesador de memoria compartida con 6 procesadores. Cada procesador dispone de una memoria cache privada de datos, que por simplificar se va a considerar que tiene sólo cuatro bloques, es de ubicación directa y que cada bloque tiene sólo dos palabras. Para mantener la coherencia usa la política de invalidación MSI, sobre una implementación snoopy

Indique las peticiones en el bus que desencadenan las siguientes operaciones y qué cambios se producen tanto en las cachés como en la memoria principal. Suponga que cada operación es independiente de las demás, por lo que se parte del estado inicial mostrado en la siguiente figura. Para simplificar la figura, el tag contiene la dirección completa y de cada dato solo se representa el último byte.

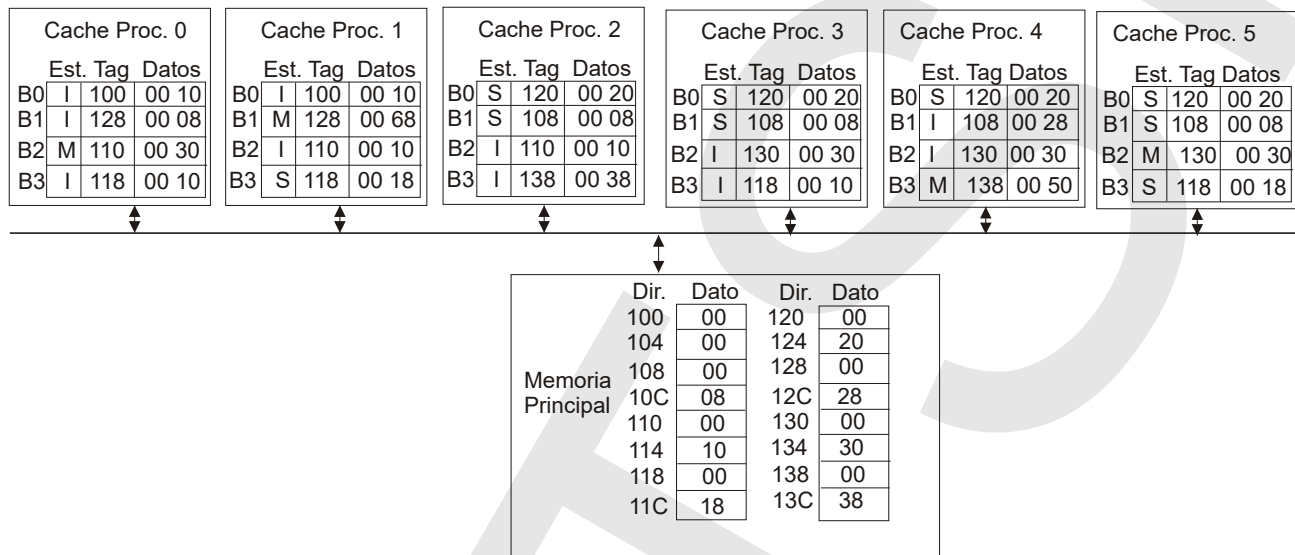


Figura 1 Multiprocesador con coherencia de caches basada en MSI

- a) Procesador 4 lee el dato de la dirección 118.
- b) Procesador 1 escribe 80 en la dirección 12C.
- c) Procesador 0 lee el dato de la dirección 128.
- d) Procesador 5 escribe 80 en la dirección 114.
- e) Procesador 5 escribe 80 en la dirección 11C.
- f) Procesador 4 escribe 80 en la dirección 118.

## SOLUCIÓN

- a) Provoca un Fallo de cache, como debe ir al Bloque B3, que está modificado, primero lo escribe en Mp, luego envía un BusRd. Mp [138]: 00, Mp [13C]: 50, P4 B3 S 118 (00 18).
- b) Provoca Acierto de cache, no genera tráfico en el bus ya que está modificado. P1 B1 M 128 (00 80).
- c) Provoca un Fallo de cache ya que está invalidado. Genera un BusRd. Como el P1 lo tiene modificado, primero lo vuelca a Mp y cambia su estado a Shared. Mp [128]: 00, Mp [12C]: 68, P0 B1 S 128 (00 68), P1 B1 S 128 (00 68).
- d) Provoca un Fallo de cache, como debe ir al Bloque B2, desaloja un bloque modificado, que primero se vuelca a Mp. Luego genera un BusRdX, que invalida la copia en P0. Como el bloque está en P0 modificado primero lo vuelca a Mp. Mp [130]: 00, Mp [134]: 30, Mp [110]: 00, Mp [114]: 30, P0 B2 I 110 (00 30), P5 M 110 (00 80).
- e) Provoca un Acierto de cache, al estar en estado Shared genera un BusInv para invalidar las otras copias en cache. P1 B3 I 118 (00 18), P5 B3 M 118 (00 80).
- f) Provoca un Fallo de cache. Como desplaza un bloque modificado, B3, primero lo escribe en Mp, luego genera un BusRdX, que invalida la copia de P1 y P5. Mp [138]: 00, Mp [13C]: 50, P1 B3 I 118 (00 18), P4 B3 M 118 (80 18), P5 B3 I 118 (00 18).